

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 3.3 — Deploying and Implementing Cloud Run and Cloud Functions Resources

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 3: Deploying and implementing a cloud solution (~25 % of the exam)**. You must know how to deploy applications to Cloud Run and Cloud Functions, configure event-driven architectures using Pub/Sub, Cloud Storage, and Eventarc, and determine where to deploy applications.

1. Cloud Run — Deploying an Application

What Is Cloud Run?

- Fully managed serverless platform for running **containers**
- Automatically scales from zero to thousands of instances
- Pay only when handling requests (per 100ms billing granularity)
- Supports any language/framework (container-based)

Deploying to Cloud Run

From a Container Image

```
# Deploy a container image to Cloud Run
gcloud run deploy my-service \
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \
  --region=us-central1 \
  --platform=managed \
  --allow-unauthenticated \
  --port=8080 \
  --memory=512Mi \
  --cpu=1 \
  --min-instances=0 \
  --max-instances=100 \
  --timeout=300 \
  --set-env-vars="DB_HOST=10.0.0.5,ENV=production" \
  --service-account=my-sa@PROJECT_ID.iam.gserviceaccount.com
```

From Source Code (Cloud Build)

```
# Deploy directly from source (Cloud Build creates the container)
gcloud run deploy my-service \
  --source=. \
  --region=us-central1 \
  --allow-unauthenticated
```

Key Deployment Parameters

Parameter	Description	Default
<code>--image</code>	Container image URL	Required
<code>--region</code>	Deployment region	Required
<code>--port</code>	Container port	8080
<code>--memory</code>	Memory allocation	512Mi
<code>--cpu</code>	CPU allocation	1
<code>--min-instances</code>	Minimum instances (0 = scale to zero)	0
<code>--max-instances</code>	Maximum instances	100
<code>--timeout</code>	Request timeout (seconds)	300 (5 min)
<code>--concurrency</code>	Max concurrent requests per instance	80
<code>--allow-unauthenticated</code>	Allow public access	Requires auth by default
<code>--no-allow-unauthenticated</code>	Require authentication	
<code>--service-account</code>	Service account for the service	Default compute SA
<code>--set-env-vars</code>	Environment variables	None
<code>--set-secrets</code>	Mount secrets from Secret Manager	None
<code>--vpc-connector</code>	Connect to a VPC	None
<code>--ingress</code>	Control inbound access (<code>all</code> , <code>internal</code> , <code>internal-and-cloud-load-balancing</code>)	<code>all</code>

Cloud Run Revisions

Each deployment creates a new **revision** (immutable snapshot of the service):

```
# List revisions
gcloud run revisions list --service=my-service --region=us-central1

# Describe a revision
gcloud run revisions describe REVISION_NAME --region=us-central1

# Delete a revision
gcloud run revisions delete REVISION_NAME --region=us-central1
```

Traffic Management

```
# Send 100% traffic to a specific revision
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00005-abc=100

# Split traffic between revisions (canary deployment)
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00004-xyz=90,my-service-00005-abc=10

# Roll back to the previous revision
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00004-xyz=100

# Gradual rollout with tags
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-tags=canary=10
```

2. Cloud Functions — Deploying an Application

What Are Cloud Functions?

- **Function as a Service (FaaS)** — event-driven, serverless functions
- Write code in supported languages, deploy, and Google manages everything
- Two generations: Gen 1 (legacy) and Gen 2 (recommended, built on Cloud Run)

Supported Runtimes

Language	Gen 1	Gen 2
Node.js	16, 18, 20	18, 20, 22
Python	3.8-3.12	3.8-3.12
Go	1.19-1.22	1.19-1.22
Java	11, 17, 21	11, 17, 21
.NET	6, 8	6, 8
Ruby	3.0-3.3	3.0-3.3
PHP	8.1-8.3	8.1-8.3

Deploying Gen 2 Cloud Functions

HTTP-Triggered Function

```
# Deploy an HTTP function
gcloud functions deploy my-http-function \
  --gen2 \
  --region=us-central1 \
  --runtime=python312 \
  --trigger-http \
  --allow-unauthenticated \
  --entry-point=hello_http \
  --memory=256Mi \
  --timeout=60s \
  --min-instances=0 \
  --max-instances=100 \
  --source=.
```

Python example (`main.py`):

```
import functions_framework

@functions_framework.http
def hello_http(request):
    name = request.args.get("name", "World")
    return f"Hello, {name}!"
```

Event-Triggered Function

```
# Deploy a Pub/Sub-triggered function
gcloud functions deploy my-pubsub-function \
  --gen2 \
  --region=us-central1 \
  --runtime=python312 \
  --trigger-topic=my-topic \
  --entry-point=process_message \
  --source=.

# Deploy a Cloud Storage-triggered function
gcloud functions deploy my-storage-function \
  --gen2 \
  --region=us-central1 \
  --runtime=python312 \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=my-bucket" \
  --entry-point=process_file \
  --source=.
```

Python Pub/Sub example (`main.py`):

```
import base64
import functions_framework
from cloudevents.http import CloudEvent

@functions_framework.cloud_event
def process_message(cloud_event: CloudEvent):
    data = base64.b64decode(cloud_event.data["message"]["data"]).decode()
    print(f"Received message: {data}")
```

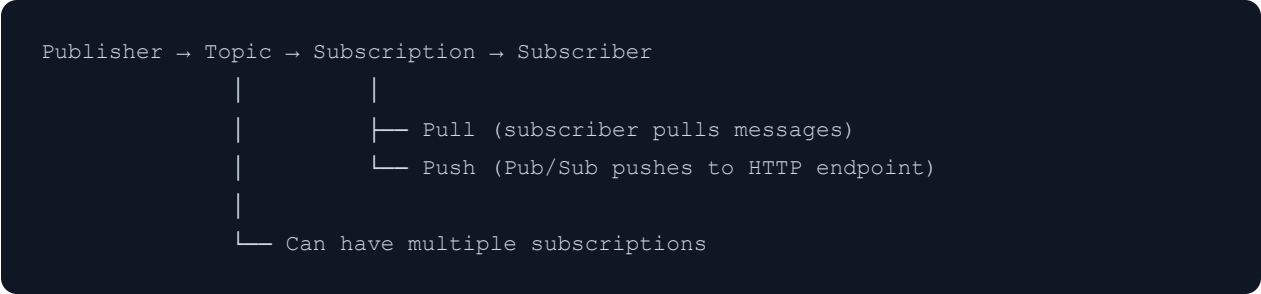
Gen 1 vs. Gen 2 Comparison

Feature	Gen 1	Gen 2
Max timeout	9 minutes	60 minutes
Max memory	8 GB	32 GB
Max vCPUs	2	8
Concurrency	1 request per instance	Up to 1,000 per instance
Traffic splitting	No	Yes (built on Cloud Run)
Min instances	Yes	Yes
Event sources	HTTP, Pub/Sub, Storage, Firestore	Same + Eventarc (100+)
Built on	Custom infra	Cloud Run

3. Event-Driven Architectures

Pub/Sub Events

Pub/Sub is Google Cloud’s messaging service for event-driven architectures:



```
# Create a Pub/Sub topic
gcloud pubsub topics create my-topic

# Create a push subscription to Cloud Run
gcloud pubsub subscriptions create my-sub \
  --topic=my-topic \
  --push-endpoint=https://my-service-xxx-uc.a.run.app

# Publish a test message
gcloud pubsub topics publish my-topic --message="Hello!"
```

Cloud Storage Object Change Notifications

Trigger functions when objects are created, deleted, or modified in Cloud Storage:

Event Type	Trigger
<code>google.cloud.storage.object.v1.finalized</code>	Object created or overwritten
<code>google.cloud.storage.object.v1.deleted</code>	Object deleted
<code>google.cloud.storage.object.v1.archived</code>	Object archived (versioned bucket)
<code>google.cloud.storage.object.v1.metadataUpdated</code>	Object metadata changed

```
# Deploy function triggered by new objects in a bucket
gcloud functions deploy process-uploads \
  --gen2 \
  --region=us-central1 \
  --runtime=nodejs20 \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=my-upload-bucket" \
  --entry-point=processFile
```

Eventarc

Eventarc is the unified event routing service for Google Cloud:

```
Event Source → Eventarc Trigger → Target (Cloud Run, Cloud Functions, GKE, Workflows)
```

Supported event sources (100+): - Cloud Storage (object changes) - Pub/Sub (messages) - Cloud Audit Logs (any audited API call) - Firebase events - Third-party sources (via Pub/Sub) - Custom events


```
# Create an Eventarc trigger for Cloud Audit Logs
gcloud eventarc triggers create my-audit-trigger \
  --destination-run-service=my-service \
  --destination-run-region=us-central1 \
  --event-filters="type=google.cloud.audit.log.v1.written" \
  --event-filters="serviceName=storage.googleapis.com" \
  --event-filters="methodName=storage.objects.create" \
  --service-account=my-trigger-sa@PROJECT_ID.iam.gserviceaccount.com

# List triggers
gcloud eventarc triggers list --location=us-central1

# Describe a trigger
gcloud eventarc triggers describe my-trigger --location=us-central1
```

Event Architecture Patterns

```
Pattern 1: Direct Trigger
Cloud Storage → Cloud Function (processes file)

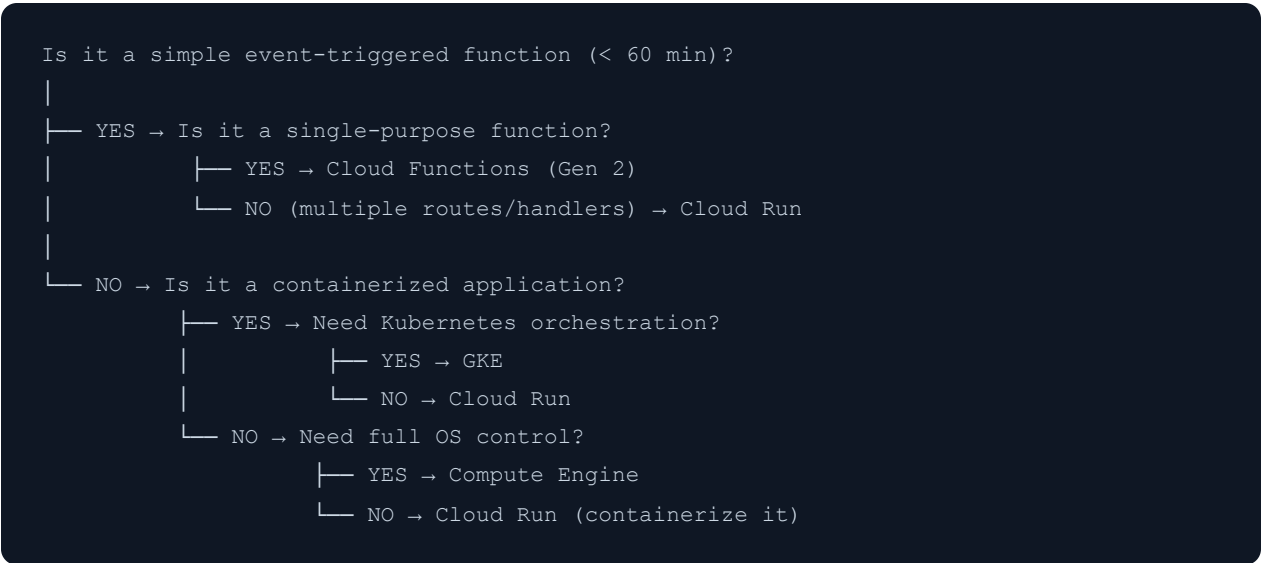
Pattern 2: Pub/Sub Decoupling
Publisher → Pub/Sub → Cloud Run (processes messages)

Pattern 3: Eventarc Routing
Cloud Audit Log → Eventarc → Cloud Run (reacts to API calls)

Pattern 4: Fan-out
Pub/Sub Topic → Subscription 1 → Cloud Function A
                → Subscription 2 → Cloud Function B
                → Subscription 3 → Cloud Run C
```

4. Determining Where to Deploy

Decision Flowchart



Cloud Run vs. Cloud Functions — When to Choose

Choose Cloud Run When	Choose Cloud Functions When
Multiple endpoints/routes	Single-purpose function
Need custom Dockerfile	Simple code in supported runtime
Need WebSocket/gRPC	Quick event processing
Team prefers containers	Team prefers writing functions
Need more than 32 GB memory	Short execution time (< 60 min)
Want traffic splitting	Simple Pub/Sub or Storage triggers
Migrating existing container	Glue code between services

Cloud Run for Anthos (GKE)

- Run Cloud Run workloads on your own GKE cluster
- Use case: Workloads that must run on dedicated infrastructure
- Benefits from GKE networking, GPU, and compliance capabilities
- Being deprecated in favor of standard Cloud Run + GKE

5. Cloud Run Configuration Deep Dive

Environment Variables and Secrets

```
# Set environment variables
gcloud run deploy my-service \
  --set-env-vars="KEY1=value1,KEY2=value2"

# Mount Secret Manager secrets as env vars
gcloud run deploy my-service \
  --set-secrets="DB_PASSWORD=my-secret:latest"

# Mount secrets as files
gcloud run deploy my-service \
  --set-secrets="/secrets/db-password=my-secret:latest"

# Update env vars on existing service
gcloud run services update my-service \
  --update-env-vars="KEY3=value3" \
  --region=us-central1
```

VPC Connectivity

```
# Create a Serverless VPC Access connector
gcloud compute networks vpc-access connectors create my-connector \
  --region=us-central1 \
  --subnet=my-subnet \
  --subnet-project=PROJECT_ID

# Deploy Cloud Run with VPC connector
gcloud run deploy my-service \
  --vpc-connector=my-connector \
  --vpc-egress=all-traffic \
  --region=us-central1
```

Ingress Settings

Setting	Description
<code>all</code>	Accept traffic from anywhere (default)
<code>internal</code>	Only from VPC, Cloud Interconnect, or VPN
<code>internal-and-cloud-load-balancing</code>	From VPC or through external Application LB

6. Managing Cloud Functions

```
# List deployed functions
gcloud functions list

# Describe a function
gcloud functions describe my-function --region=us-central1

# View function logs
gcloud functions logs read my-function --region=us-central1

# Delete a function
gcloud functions delete my-function --region=us-central1

# Call an HTTP function
gcloud functions call my-http-function \
  --data='{"name": "World"}' \
  --region=us-central1

# Update function configuration
gcloud functions deploy my-function \
  --gen2 \
  --region=us-central1 \
  --runtime=python312 \
  --memory=512Mi \
  --timeout=120s \
  --update-env-vars="NEW_VAR=value"
```

7. Cloud Scheduler Integration

- Cloud Scheduler sends HTTP requests or Pub/Sub messages on a cron schedule
- Use cases: scheduled Cloud Run jobs, periodic Cloud Functions invocations, batch pipelines

```
# Creating a Cloud Scheduler job to invoke Cloud Run
gcloud scheduler jobs create http my-job \
  --location=us-central1 \
  --schedule="0 9 * * 1" \
  --uri="https://my-service-xxx-uc.a.run.app/run" \
  --http-method=POST \
  --message-body='{"key":"value"}' \
  --oidc-service-account-email=MY_SA@PROJECT.iam.gserviceaccount.com \
  --oidc-token-audience="https://my-service-xxx-uc.a.run.app"
# Via Pub/Sub to Cloud Functions
gcloud scheduler jobs create pubsub my-pubsub-job \
  --location=us-central1 \
  --schedule="*/5 * * * *" \
  --topic=MY_TOPIC \
  --message-body="trigger"
gcloud scheduler jobs list --location=us-central1
gcloud scheduler jobs run my-job --location=us-central1 # manual trigger for testing
```

- OIDC token is required for Cloud Run (authenticated services); the scheduler SA must have `roles/run.invoker`

8. Service-to-Service Authentication

- When Cloud Run service A calls Cloud Run service B (which requires auth):
 1. Service A's service account must have `roles/run.invoker` on service B
 2. Service A fetches an ID token from the metadata server with the audience set to service B's URL

```
import google.auth.transport.requests
import google.oauth2.id_token
audience = "https://service-b-xxx-uc.a.run.app"
request = google.auth.transport.requests.Request()
token = google.oauth2.id_token.fetch_id_token(request, audience)
# Then set Authorization: Bearer <token> in the outbound request
```

- **Exam tip:** OIDC ID tokens (for identity verification) vs. OAuth2 access tokens (for GCP API calls) — Cloud Run auth uses OIDC

9. Custom Domains and Ingress

- Cloud Run: map custom domain via Google Cloud Console or:

```
gcloud beta run domain-mappings create --service=MY_SERVICE --domain=api.example.com --region=us-central1
gcloud beta run domain-mappings list --region=us-central1
```

- Requires DNS CNAME verification; Google manages the TLS certificate automatically
- Ingress options (controls who can reach the service):
 - `all` : any internet traffic
 - `internal` : only from same VPC, Shared VPC, or Cloud Run in same project
 - `internal-and-cloud-load-balancing` : internal + traffic through Google Cloud external LB

```
gcloud run services update MY_SERVICE --ingress=internal --region=us-central1
```

- To expose an internal-only Cloud Run service publicly: put it behind an external HTTPS LB with Cloud Run NEG backend

Exam Practice Questions

1. **You need to deploy a Python function that processes images uploaded to Cloud Storage. The processing takes about 30 seconds per image. Which service should you use and how?**
 - Answer: **Cloud Functions Gen 2** with a Cloud Storage trigger on the `finalized` event. Deploy with `--trigger-event-filters="type=google.cloud.storage.object.v1.finalized"`.
2. **Your team has a containerized Go application with 5 HTTP endpoints. They want serverless scaling with no Kubernetes overhead. Where should they deploy?**
 - Answer: **Cloud Run** — Supports containers with multiple endpoints, serverless scaling, and no cluster management.
3. **You want to perform a canary release, sending 10% of traffic to a new version of your Cloud Run service. How?**
 - Answer: Deploy the new version, then use `gcloud run services update-traffic --to-revisions=old=90,new=10`.
4. **A Cloud Run service needs to access a Cloud SQL instance over a private IP. What do you need to configure?**

- Answer: Create a **Serverless VPC Access connector** and deploy the Cloud Run service with `--vpc-connector` and `--vpc-egress=private-ranges-only` (or `all-traffic`).
5. **Your function needs to react when any VM is created in your project. Which event source and service should you use?**
- Answer: Use **Eventarc** with a Cloud Audit Log trigger: filter on `type=google.cloud.audit.log.v1.written`, `serviceName=compute.googleapis.com`, `methodName=v1.compute.instances.insert`. Route to Cloud Run or Cloud Functions.
6. **What is the maximum request timeout for Cloud Functions Gen 2?**
- Answer: **60 minutes** (same as Cloud Run, since Gen 2 is built on Cloud Run).
7. **You need your Cloud Run service to only accept traffic from within your VPC and through your external load balancer. Which ingress setting should you use?**
- Answer: `--ingress=internal-and-cloud-load-balancing`
-

Glossary

Apache Beam — An open-source, unified programming model for defining batch and streaming data processing pipelines. Google Dataflow is a fully managed runner for Apache Beam pipelines.

Artifact Registry — GCP's fully managed service for storing and managing container images and other build artifacts. Serves as the image source for Cloud Run and GKE deployments.

Audit Log (Cloud Audit Log) — A GCP log that records administrative actions (Admin Activity) and data access operations on GCP resources. Used as an Eventarc event source to trigger Cloud Run or Cloud Functions on any audited API call.

Cloud Build — GCP's fully managed CI/CD build service. Used with `gcloud run deploy --source=.` to automatically build a container image from source code before deploying to Cloud Run.

Cloud Functions — GCP's serverless Function as a Service (FaaS) offering that executes event-driven code without provisioning or managing servers. Supports Gen 1 (legacy) and Gen 2 (recommended, built on Cloud Run).

Cloud Run — GCP's fully managed serverless platform for running containers. Automatically scales from zero to thousands of instances based on incoming requests; billing is per 100 ms of execution.

Cloud Run for Anthos — A now-deprecated feature that ran Cloud Run workloads on GKE clusters, providing serverless container execution on dedicated infrastructure.

Cloud Scheduler — GCP's fully managed cron job service that sends HTTP requests or Pub/Sub messages on a configurable schedule. Used to invoke Cloud Run services or Cloud Functions periodically.

CloudEvent — An open specification (CNCF) for describing event data in a common format. Cloud Functions Gen 2 and Eventarc use the CloudEvents format for event payloads.

CNAME (Canonical Name) — A DNS record type that aliases one domain name to another. Required when mapping a custom domain to a Cloud Run service.

Concurrency — The number of simultaneous requests a single Cloud Run instance can handle. Cloud Run default is 80; Cloud Functions Gen 1 is 1 (one request per instance); Gen 2 supports up to 1,000.

Container — A lightweight, portable unit of software packaging an application and its dependencies. Cloud Run executes containers directly; any language or framework is supported.

CRON schedule — A time-based expression (e.g., `0 9 * * 1`) defining when Cloud Scheduler jobs fire. Uses standard Unix cron syntax.

Custom Domain — A user-owned domain name (e.g., `api.example.com`) mapped to a Cloud Run service via domain mappings. Google automatically provisions and renews the TLS certificate.

Dead Letter Topic — A Pub/Sub topic that receives messages that could not be successfully processed after a configured maximum number of delivery attempts.

Dockerfile — A text file containing instructions for building a container image. Cloud Run supports any valid Dockerfile; Cloud Functions manages containerization automatically.

Entry Point — In Cloud Functions, the name of the specific function within the deployed source code file that is invoked when the trigger fires. Specified with `--entry-point`.

Environment Variable — A key-value pair passed to a running container or function at runtime to configure behavior without modifying code. Set with `--set-env-vars` in Cloud Run and Cloud Functions.

Eventarc — GCP's unified event routing service that delivers events from 100+ Google Cloud sources (Cloud Audit Logs, Pub/Sub, Cloud Storage, Firebase, etc.) to Cloud Run, Cloud Functions, GKE, and Workflows.

FaaS (Function as a Service) — A serverless compute model in which individual functions are deployed and executed in response to events without managing servers. Cloud Functions is GCP's FaaS offering.

Fan-out — An event architecture pattern in which a single Pub/Sub topic has multiple subscriptions, each routing messages to a different downstream consumer (Cloud Function, Cloud Run service, etc.).

GCP (Google Cloud Platform) — Google's suite of cloud computing services, including compute, storage, networking, databases, analytics, and machine learning.

gcloud — The primary command-line tool for interacting with GCP services, part of the Google Cloud SDK.

GKE (Google Kubernetes Engine) — GCP's managed Kubernetes service. Eventarc can route events to workloads running on GKE clusters.

Go — A statically typed, compiled programming language developed by Google. Supported as a Cloud Functions and Dataflow runtime.

gRPC — A high-performance Remote Procedure Call framework. Cloud Run natively supports gRPC in addition to HTTP, unlike Cloud Functions Gen 1.

IAM (Identity and Access Management) — GCP's system for controlling who can perform what actions on which resources. Controls access to Cloud Run services and determines which service accounts can invoke functions.

ID Token (OIDC Token) — A signed token issued by Google's identity platform that asserts the identity of a service account. Used for authenticated Cloud Run service-to-service calls (set in `Authorization: Bearer` header).

Ingress (Cloud Run) — A Cloud Run setting that controls which network sources can send requests to a service: `all` (public internet), `internal` (VPC/Interconnect/VPN only), or `internal-and-cloud-load-balancing`.

Java — A widely used, object-oriented programming language. Supported as a Cloud Functions runtime in versions 11, 17, and 21.

Min Instances — A Cloud Run and Cloud Functions configuration that keeps a specified number of instances warm at all times, eliminating cold starts. Setting to 0 allows scale-to-zero.

Max Instances — The upper limit on the number of Cloud Run or Cloud Functions instances that can run concurrently, providing cost control and preventing downstream service overload.

Metadata Server — An internal GCP service (accessible at `http://metadata.google.internal`) from which Cloud Run containers and VMs retrieve instance metadata and identity tokens.

Microservice — An architectural pattern in which an application is built as a collection of small, independently deployable services communicating over APIs. Cloud Run is commonly used to host microservices.

Node.js — A JavaScript runtime for server-side applications. Supported as a Cloud Functions and Cloud Run runtime.

OAuth2 Access Token — A token used to authenticate calls to GCP service APIs (e.g., Cloud Storage). Distinct from OIDC ID tokens, which are used for Cloud Run service-to-service authentication.

OIDC (OpenID Connect) — An identity layer built on OAuth 2.0. Cloud Run and Cloud Scheduler use OIDC tokens to authenticate HTTP requests to secured Cloud Run services.

PHP — A server-side scripting language. Supported as a Cloud Functions runtime in versions 8.1–8.3.

Pub/Sub — GCP's fully managed, asynchronous messaging service. Used as an event source for Cloud Functions and Cloud Run, and as the underlying delivery mechanism for Eventarc triggers.

Pull Subscription — A Pub/Sub subscription model in which the subscriber explicitly calls the service to retrieve messages. Used for applications that control their own consumption rate.

Push Subscription — A Pub/Sub subscription model in which the service delivers messages by making HTTP POST requests to a configured endpoint (e.g., a Cloud Run URL or Cloud Functions URL).

Python — A general-purpose programming language. Supported as a Cloud Functions runtime in versions 3.8–3.12 and used in code examples throughout this chapter.

Revision (Cloud Run) — An immutable snapshot of a Cloud Run service's configuration and container image, created with each deployment. Traffic can be split across revisions for canary releases or rollbacks.

Ruby — A dynamic programming language. Supported as a Cloud Functions runtime in versions 3.0–3.3.

Scale to Zero — A Cloud Run and Cloud Functions capability that terminates all instances when there are no incoming requests, reducing costs to zero during idle periods.

Secret Manager — GCP's service for securely storing, accessing, and managing sensitive configuration data (API keys, passwords, certificates). Cloud Run mounts secrets as environment variables or files via `--set-secrets`.

Serverless — A cloud execution model in which the provider automatically manages infrastructure provisioning, scaling, and maintenance. Cloud Run and Cloud Functions are GCP's primary serverless compute products.

Serverless VPC Access — A GCP feature that allows Cloud Run services and Cloud Functions to connect to resources within a VPC network (e.g., Cloud SQL on private IP, Memorystore) using a VPC connector.

Service Account — A GCP IAM identity used by applications and services (rather than humans) to authenticate API calls. Cloud Run services run under a specified service account for fine-grained access control.

Storage Transfer Service — A GCP managed service for large-scale, scheduled data transfers between Cloud Storage buckets, from AWS S3, or from HTTP/HTTPS sources.

Tag (Cloud Run) — A named pointer to a specific Cloud Run revision, accessible via a stable URL prefix (e.g., `canary---my-service-xxx.run.app`). Used for staged traffic rollouts.

Timeout — The maximum duration a Cloud Run request or Cloud Functions invocation can run before being terminated. Cloud Run supports up to 3,600 seconds; Cloud Functions Gen 2

supports up to 3,600 seconds (60 minutes).

TLS (Transport Layer Security) — The cryptographic protocol securing HTTPS connections. Cloud Run and custom domain mappings use Google-managed TLS certificates.

Topic (Pub/Sub) — A named resource to which publishers send messages. One topic can have multiple subscriptions, each receiving a copy of every published message.

Traffic Splitting — A Cloud Run feature that distributes incoming requests across multiple revisions by percentage (e.g., 90% to stable, 10% to canary). Not available in Cloud Functions Gen 1.

Trigger (Cloud Functions / Eventarc) — A configuration that specifies the event type and source that causes a function or Cloud Run service to be invoked (e.g., Pub/Sub message, Cloud Storage object finalized).

vCPU — A virtual CPU allocated to a Cloud Run instance or Cloud Functions instance. Cloud Functions Gen 2 supports up to 8 vCPUs per instance; Gen 1 supports up to 2.

VPC (Virtual Private Cloud) — GCP's global, software-defined private network. Cloud Run can connect to VPC resources (e.g., Cloud SQL, Memorystore) via a Serverless VPC Access connector.

VPC Connector — A Serverless VPC Access resource that provides a network bridge between Cloud Run or Cloud Functions and a VPC network, enabling access to private IP resources.

WebSocket — A protocol providing full-duplex communication channels over a single TCP connection. Supported by Cloud Run but not by Cloud Functions, making Cloud Run the appropriate choice for real-time applications.

Workflows — GCP's fully managed service for orchestrating sequences of HTTP-based services and GCP APIs. Eventarc can route events to Workflows as a target.

.NET — Microsoft's cross-platform development framework. Supported as a Cloud Functions runtime in versions 6 and 8.